Analyzing Customer Orders Using Python

Course-End Project

Project Objective

This project analyzes customer orders for an e-commerce company using Python's built-in data structures and control structures.

The analysis focuses on:

- Storing and organizing customer order data
- Classifying products into different categories
- Calculating total spending for each customer
- Classifying customers based on their spending
- Calculating revenue for each product category
- Identifying unique products and product categories
- Finding customers who purchased electronics
- Identifying the top three highest-spending customers
- Analyzing customers who purchased products from multiple categories
- Generating useful business insights from customer purchasing patterns

Python Concepts Used

This project demonstrates the use of:

- Lists
- Tuples
- Dictionaries
- Sets
- Loops
- Conditional statements
- List comprehensions
- Sorting

In [33]:
```
# Customer names
customers = [
    "Alice",
    "Bob",
    "Charlie",
    "David",
    "Emma",
    "Frank",
    "Grace",
    "Henry",
    "Isha"
]
```

```
# Customer order data
# Each order is stored as:
# (Customer Name, Product, Price, Category)

orders = [
    ("Alice", "Laptop", 800, "Electronics"),
    ("Alice", "Headphones", 50, "Electronics"),
    ("Alice", "T-Shirt", 30, "Clothing"),

    ("Bob", "Smartphone", 600, "Electronics"),
    ("Bob", "Jeans", 60, "Clothing"),

    ("Charlie", "Microwave", 120, "Home Essentials"),
    ("Charlie", "T-Shirt", 25, "Clothing"),

    ("David", "Laptop", 800, "Electronics"),
    ("David", "Smartwatch", 150, "Electronics"),
    ("David", "Jacket", 90, "Clothing"),

    ("Emma", "Sofa", 450, "Home Essentials"),
    ("Emma", "Bedsheet", 40, "Home Essentials"),

    ("Frank", "Headphones", 50, "Electronics"),
    ("Frank", "T-Shirt", 25, "Clothing"),

    ("Grace", "Refrigerator", 700, "Home Essentials"),
    ("Grace", "Jeans", 60, "Clothing"),

    ("Henry", "Keyboard", 70, "Electronics"),
    ("Henry", "Mouse", 30, "Electronics"),
    ("Henry", "T-Shirt", 25, "Clothing"),

    ("Isha", "T-Shirt", 25, "Clothing")
]
```

In [34]:

```
# Create a dictionary to store products ordered by each customer

customer_orders = {}

for customer, product, price, category in orders:
    if customer not in customer_orders:
        customer_orders[customer] = []

    customer_orders[customer].append(product)

print("Customer Orders Dictionary:")
for customer, products in customer_orders.items():
    print(f"{customer}: {products}")
```

```
Customer Orders Dictionary:
Alice: ['Laptop', 'Headphones', 'T-Shirt']
Bob: ['Smartphone', 'Jeans']
Charlie: ['Microwave', 'T-Shirt']
David: ['Laptop', 'Smartwatch', 'Jacket']
Emma: ['Sofa', 'Bedsheet']
Frank: ['Headphones', 'T-Shirt']
Grace: ['Refrigerator', 'Jeans']
Henry: ['Keyboard', 'Mouse', 'T-Shirt']
Isha: ['T-Shirt']
```

In [35]:
```python
# Create a dictionary mapping each product to its category

product_category = {
    "Laptop": "Electronics",
    "Headphones": "Electronics",
    "Smartphone": "Electronics",
    "Smartwatch": "Electronics",
    "Keyboard": "Electronics",
    "Mouse": "Electronics",
    "T-Shirt": "Clothing",
    "Jeans": "Clothing",
    "Jacket": "Clothing",
    "Microwave": "Home Essentials",
    "Sofa": "Home Essentials",
    "Bedsheet": "Home Essentials",
    "Refrigerator": "Home Essentials"
}

# Create a set containing unique product categories
unique_categories = set(product_category.values())

print("Product Categories:")
for product, category in product_category.items():
    print(f"{product}: {category}")

print("\nUnique Product Categories:")
print(unique_categories)
```

```
Product Categories:
Laptop: Electronics
Headphones: Electronics
Smartphone: Electronics
Smartwatch: Electronics
Keyboard: Electronics
Mouse: Electronics
T-Shirt: Clothing
Jeans: Clothing
Jacket: Clothing
Microwave: Home Essentials
Sofa: Home Essentials
Bedsheet: Home Essentials
Refrigerator: Home Essentials

Unique Product Categories:
{'Home Essentials', 'Electronics', 'Clothing'}
```

In [36]:
```python
# Calculate total spending for each customer

customer_spending = {}

for customer, product, price, category in orders:
    if customer not in customer_spending:
        customer_spending[customer] = 0

    customer_spending[customer] += price


# Classify customers based on their total spending

customer_classification = {}
```

```
for customer, total in customer_spending.items():
    if total > 100:
        customer_classification[customer] = "High-value"
    elif total >= 50:
        customer_classification[customer] = "Moderate"
    else:
        customer_classification[customer] = "Low-value"


# Display customer spending and classification

print("Customer Spending and Classification")
print("-" * 45)

for customer in customers:
    print(
        f"{customer}: ${customer_spending[customer]:.2f} "
        f"- {customer_classification[customer]}"
    )
```

```
Customer Spending and Classification
---------------------------------------------
Alice: $880.00 - High-value
Bob: $660.00 - High-value
Charlie: $145.00 - High-value
David: $1040.00 - High-value
Emma: $490.00 - High-value
Frank: $75.00 - Moderate
Grace: $760.00 - High-value
Henry: $125.00 - High-value
Isha: $25.00 - Low-value
```

In [37]:
```
# Calculate total revenue for each product category

category_revenue = {}

for customer, product, price, category in orders:
    if category not in category_revenue:
        category_revenue[category] = 0

    category_revenue[category] += price


# Display category-wise revenue

print("Revenue by Product Category")
print("-" * 35)

for category, revenue in category_revenue.items():
    print(f"{category}: ${revenue:.2f}")
```

```
Revenue by Product Category
-----------------------------------
Electronics: $2550.00
Clothing: $340.00
Home Essentials: $1310.00
```

In [38]:
```
# Extract unique products from all customer orders

unique_products = set()
```

```
for customer, product, price, category in orders:
    unique_products.add(product)

print("Unique Products:")
for product in sorted(unique_products):
    print(product)

print("\nTotal Number of Unique Products:", len(unique_products))
```

```
Unique Products:
Bedsheet
Headphones
Jacket
Jeans
Keyboard
Laptop
Microwave
Mouse
Refrigerator
Smartphone
Smartwatch
Sofa
T-Shirt

Total Number of Unique Products: 13
```

In [39]:
```
# Find customers who purchased electronics

electronics_customers = [
    customer
    for customer in customers
    if any(category == "Electronics" for cust, product, price, category in order
]

print("Customers who purchased Electronics:")
for customer in electronics_customers:
    print(customer)
```

```
Customers who purchased Electronics:
Alice
Bob
David
Frank
Henry
```

In [40]:
```
# Find the top three highest-spending customers

top_customers = sorted(
    customer_spending.items(),
    key=lambda item: item[1],
    reverse=True
)

print("Top 3 Highest-Spending Customers:")
print("-" * 40)

for rank, (customer, total) in enumerate(top_customers[:3], start=1):
    print(f"{rank}. {customer}: ${total:.2f}")
```

```
Top 3 Highest-Spending Customers:
--------------------------------------
1. David: $1040.00
2. Alice: $880.00
3. Grace: $760.00
```

In [41]:
```python
# Create sets of customers for each product category

electronics_customers_set = set()
clothing_customers_set = set()
home_essentials_customers_set = set()

for customer, product, price, category in orders:
    if category == "Electronics":
        electronics_customers_set.add(customer)
    elif category == "Clothing":
        clothing_customers_set.add(customer)
    elif category == "Home Essentials":
        home_essentials_customers_set.add(customer)


print("Electronics Customers:")
print(electronics_customers_set)

print("\nClothing Customers:")
print(clothing_customers_set)

print("\nHome Essentials Customers:")
print(home_essentials_customers_set)
```

```
Electronics Customers:
{'Frank', 'Bob', 'Alice', 'Henry', 'David'}

Clothing Customers:
{'Frank', 'Bob', 'Isha', 'Grace', 'Charlie', 'Alice', 'Henry', 'David'}

Home Essentials Customers:
{'Grace', 'Emma', 'Charlie'}
```

In [42]:
```python
# Find customers who purchased both Electronics and Clothing

electronics_and_clothing = (
    electronics_customers_set & clothing_customers_set
)

print("Customers who purchased both Electronics and Clothing:")
print(electronics_and_clothing)
```

```
Customers who purchased both Electronics and Clothing:
{'Frank', 'Bob', 'Alice', 'Henry', 'David'}
```

In [43]:
```python
# Find customers who purchased products from multiple categories

customer_categories = {}

for customer, product, price, category in orders:
    if customer not in customer_categories:
        customer_categories[customer] = set()

    customer_categories[customer].add(category)
```

```
print("Customers who purchased from multiple categories:")
print("-" * 50)

for customer, categories in customer_categories.items():
    if len(categories) > 1:
        print(f"{customer}: {categories}")
```

```
Customers who purchased from multiple categories:
--------------------------------------------------
Alice: {'Electronics', 'Clothing'}
Bob: {'Electronics', 'Clothing'}
Charlie: {'Home Essentials', 'Clothing'}
David: {'Electronics', 'Clothing'}
Frank: {'Electronics', 'Clothing'}
Grace: {'Home Essentials', 'Clothing'}
Henry: {'Electronics', 'Clothing'}
```

In [44]:
```
# Count how many times each product was purchased

product_frequency = {}

for customer, product, price, category in orders:
    if product not in product_frequency:
        product_frequency[product] = 0

    product_frequency[product] += 1


# Sort products by purchase frequency

most_purchased_products = sorted(
    product_frequency.items(),
    key=lambda item: item[1],
    reverse=True
)

print("Product Purchase Frequency")
print("-" * 35)

for product, count in most_purchased_products:
    print(f"{product}: {count} purchase(s)")
```

```
Product Purchase Frequency
-----------------------------------
T-Shirt: 5 purchase(s)
Laptop: 2 purchase(s)
Headphones: 2 purchase(s)
Jeans: 2 purchase(s)
Smartphone: 1 purchase(s)
Microwave: 1 purchase(s)
Smartwatch: 1 purchase(s)
Jacket: 1 purchase(s)
Sofa: 1 purchase(s)
Bedsheet: 1 purchase(s)
Refrigerator: 1 purchase(s)
Keyboard: 1 purchase(s)
Mouse: 1 purchase(s)
```

In [46]:
```
# Display a final summary of customer spending and classification
```

```
print("Customer Purchase Summary")
print("=" * 50)

for customer in customers:
    total = customer_spending[customer]
    classification = customer_classification[customer]

    print(f"Customer: {customer}")
    print(f"Total Spending: ${total:.2f}")
    print(f"Classification: {classification}")
    print("-" * 50)
```

```
Customer Purchase Summary
==================================================
Customer: Alice
Total Spending: $880.00
Classification: High-value
--------------------------------------------------
Customer: Bob
Total Spending: $660.00
Classification: High-value
--------------------------------------------------
Customer: Charlie
Total Spending: $145.00
Classification: High-value
--------------------------------------------------
Customer: David
Total Spending: $1040.00
Classification: High-value
--------------------------------------------------
Customer: Emma
Total Spending: $490.00
Classification: High-value
--------------------------------------------------
Customer: Frank
Total Spending: $75.00
Classification: Moderate
--------------------------------------------------
Customer: Grace
Total Spending: $760.00
Classification: High-value
--------------------------------------------------
Customer: Henry
Total Spending: $125.00
Classification: High-value
--------------------------------------------------
Customer: Isha
Total Spending: $25.00
Classification: Low-value
--------------------------------------------------
```

In [48]:

```
# Calculate overall statistics

total_customers = len(customers)
total_orders = len(orders)
total_revenue = sum(customer_spending.values())
total_unique_products = len(unique_products)
total_categories = len(unique_categories)
total_electronics_customers = len(electronics_customers)
total_multiple_category_customers = sum(
```

```
    1 for categories in customer_categories.values()
    if len(categories) > 1
)

print("OVERALL STATISTICS")
print("=" * 50)

print(f"Total Number of Customers: {total_customers}")
print(f"Total Number of Orders: {total_orders}")
print(f"Total Revenue: ${total_revenue:.2f}")
print(f"Total Unique Products: {total_unique_products}")
print(f"Total Product Categories: {total_categories}")
print(f"Customers Who Purchased Electronics: {total_electronics_customers}")
print(
    f"Customers Who Purchased from Multiple Categories: "
    f"{total_multiple_category_customers}"
)
print(f"Highest Revenue Category: {highest_revenue_category}")
print(f"Most Frequently Purchased Product: {most_frequent_product}")
```

```
OVERALL STATISTICS
==================================================
Total Number of Customers: 9
Total Number of Orders: 20
Total Revenue: $4200.00
Total Unique Products: 13
Total Product Categories: 3
Customers Who Purchased Electronics: 5
Customers Who Purchased from Multiple Categories: 7
Highest Revenue Category: Electronics
Most Frequently Purchased Product: T-Shirt
```

Business Insights

Based on the analysis of customer orders, the following business insights were identified:

1. Customer Classification

The analysis identified 7 high-value customers, 1 moderate customer, and 1 low-value customer. This shows that most customers in the dataset have relatively high spending levels.

High-value customers can be targeted with loyalty programs, personalized offers, and exclusive promotions to encourage continued purchases.

2. Category-wise Revenue

Electronics generated the highest revenue with total sales of **$2,550.00**.

Home Essentials generated **$1,310.00**, while Clothing generated **$340.00**.

This indicates that Electronics is the strongest-performing category in terms of revenue. The company should ensure sufficient inventory of electronic products and consider targeted marketing campaigns for this category.

3. Most Frequently Purchased Product

The **T-Shirt** was the most frequently purchased product, with **5 purchases**.

This suggests consistent demand for T-Shirts and indicates that maintaining sufficient stock could help avoid inventory shortages.

4. Top Spending Customers

The three highest-spending customers were:

1. **David** — $1,040.00
2. **Alice** — $880.00
3. **Grace** — $760.00

These customers can be considered important high-value customers and could be targeted with personalized marketing and loyalty rewards.

5. Cross-Category Purchasing Behavior

Five customers — **Alice, Bob, David, Frank, and Henry** — purchased products from both Electronics and Clothing.

This indicates an opportunity for cross-selling. For example, customers purchasing electronic products could be offered relevant clothing promotions or bundled offers.

6. Overall Business Recommendation

The business should focus on retaining high-value customers, maintaining adequate inventory for frequently purchased products, and using cross-category purchasing patterns to create targeted marketing campaigns.

The category-wise revenue analysis also suggests that Electronics should receive particular attention because it generated the highest revenue in the dataset.

Conclusion

This project successfully analyzed customer order data using Python's built-in data structures and control structures.

Lists and tuples were used to store customer and order information. Dictionaries were used to organize customer orders, product categories, customer spending, and category-wise revenue. Sets were used to identify unique products, unique categories, and customers purchasing products across different categories.

Loops and conditional statements were used to calculate customer spending and classify customers as high-value, moderate, or low-value buyers. List comprehension was used to identify customers who purchased electronics, while sorting was used to determine the top three highest-spending customers.

The analysis showed that Electronics generated the highest revenue at $2,550.00, while T-Shirt was the most frequently purchased product with 5 purchases. David, Alice, and Grace were the top three highest-spending customers.

Overall, the project demonstrates how Python data structures and control structures can be applied to analyze e-commerce customer orders and generate useful business insights for marketing and inventory management.

In [49]:
```python
# Demonstrate retrieving and modifying customer records using dictionary methods

print("Dictionary Methods Demonstration")
print("=" * 50)

# Retrieve a customer's order list using the get() method
alice_orders = customer_orders.get("Alice")

print("Alice's orders before modification:")
print(alice_orders)

# Modify Alice's record by adding another product
customer_orders["Alice"].append("Mouse")

print("\nAlice's orders after modification:")
print(customer_orders.get("Alice"))

# Display all customer records using the items() method
print("\nCustomer records:")
for customer, products in customer_orders.items():
    print(f"{customer}: {products}")
```

```
Dictionary Methods Demonstration
==================================================
Alice's orders before modification:
['Laptop', 'Headphones', 'T-Shirt']

Alice's orders after modification:
['Laptop', 'Headphones', 'T-Shirt', 'Mouse']

Customer records:
Alice: ['Laptop', 'Headphones', 'T-Shirt', 'Mouse']
Bob: ['Smartphone', 'Jeans']
Charlie: ['Microwave', 'T-Shirt']
David: ['Laptop', 'Smartwatch', 'Jacket']
Emma: ['Sofa', 'Bedsheet']
Frank: ['Headphones', 'T-Shirt']
Grace: ['Refrigerator', 'Jeans']
Henry: ['Keyboard', 'Mouse', 'T-Shirt']
Isha: ['T-Shirt']
```

In [ ]: